

GOT

بوابة الأهداف — Gateway of Goals

Code Study & Defense Questions

Football Scouting Platform

100 Questions with Model Answers

PHP · SQL · JavaScript · HTML / CSS

25 questions per section

Contents

Section 1 — PHP (Backend API, authentication, security) — Q1–Q25

Section 2 — SQL (Database schema & queries) — Q26–Q50

Section 3 — JavaScript (Client logic, fetch, DOM) — Q51–Q75

Section 4 — HTML & CSS (Markup, design system, RTL) — Q76–Q100

Each question has a short model answer to help you prepare for the lecturer's discussion of the code. Answers describe how THIS project actually implements the feature.

Covers `api/config.php`, the auth endpoints (`login.php`, `register-player.php`, `admin-login.php`), `search-players.php`, `upload-video.php`, and `verify-club.php`.

1. How does `getDB()` avoid opening a new database connection on every call?

✓ It uses a `static $pdo = null;` variable inside the function. The PDO object is created only when it is still `null`, so the same connection is reused for the rest of the request (the singleton pattern).

2. What three PDO attributes are set when the connection is created, and why?

✓ `ATTR_ERRMODE = ERRMODE_EXCEPTION` (throw exceptions on errors), `ATTR_DEFAULT_FETCH_MODE = FETCH_ASSOC` (return associative arrays), and `ATTR_EMULATE_PREPARES = false` (use real native prepared statements instead of emulated ones).

3. Why is `ATTR_EMULATE_PREPARES => false` important for security?

✓ It forces the database driver to send the SQL and the parameters separately (true server-side prepared statements). This gives stronger protection against SQL injection and ensures correct data typing.

4. How does the project protect every query against SQL injection?

✓ All queries use `$db->prepare(...)` with `?` placeholders and pass user values through `$stmt->execute(...)`. User input is never concatenated directly into the SQL string.

5. What does the `: never` return type on `jsonSuccess()` / `jsonError()` mean?

✓ It is a PHP 8.1 return type declaring the function never returns normally — it always ends the script (both call `exit;`). It tells the engine and developers that no code runs after the call.

6. How are passwords stored and verified?

✓ On registration the password is hashed with `password_hash($password, PASSWORD_BCRYPT)`. On login it is checked with `password_verify($password, $user['password_hash'])`. Plain passwords are never stored.

7. Why is `session_regenerate_id(true)` called right after a successful login?

✓ To prevent **session fixation** attacks — a new session ID is issued once the user is authenticated, so any ID an attacker may have planted before login becomes invalid.

8. How does `requireSession($role)` enforce authentication and authorization?

✓ If `$_SESSION['user_id']` is empty it returns 401 (Authentication required). If a role is given and `$_SESSION['role']` doesn't match, it returns 403 (Insufficient permissions). Otherwise it returns the session.

9. What is the difference between `requireSession()` and `requireSessionMulti()`?

✓ `requireSession` checks for exactly one role; `requireSessionMulti` accepts an array of roles and uses `in_array($_SESSION['role'], $roles, true)` to allow any of several roles.

10. How does the admin login read the request and protect the secret key from timing attacks?

✓ It reads the JSON body and compares the submitted secret with `hash_equals(ADMIN_SECRET, $secret)`, a constant-time comparison that prevents attackers from guessing the secret by measuring response times.

11. How is the raw request body parsed in the API endpoints?

✓ With `json_decode(file_get_contents('php://input'), true)` — it reads the raw POST body stream and decodes the JSON into an associative array.

12. How is the email validated server-side?

✓ With `filter_var($email, FILTER_VALIDATE_EMAIL)`; URLs are validated with `FILTER_VALIDATE_URL`. These reject malformed input before touching the database.

13. Why does player registration wrap the inserts in a transaction?

✓ A player needs rows in three tables (`users`, `player_profiles`, `player_skills`). `beginTransaction()` / `commit()` make them atomic; if any insert fails, `rollback()` undoes all of them so no half-created account remains.

14. How does the code get the ID of a newly inserted row, and link the profile to its user?

✓ With `$db->lastInsertId()` after each insert — the new `user_id` is then passed into the `player_profiles` insert, and that profile's id into `player_skills`.

15. How are duplicate email registrations handled, including the race-condition case?

✓ First it does a `SELECT` to check the email and returns HTTP 409 if found. If two requests slip through simultaneously, the catch block inspects `$e->getCode() === '23000'` (a unique-key violation) and also returns 409.

16. In `upload-video.php`, how is the uploaded file's type validated and why not trust the browser?

✓ It inspects the real content with `finfo(FILEINFO_MIME_TYPE)` and checks it against `ALLOWED_VIDEO_TYPES`. The browser-supplied MIME type can be spoofed, so the server detects it from the file bytes itself.

17. How is a safe, unique filename generated for an uploaded video?

✓ `bin2hex(random_bytes(16))` creates a random 32-char name, and the extension is cleaned with `sanitizeFilename()` (a regex that strips anything not `[a-zA-Z0-9._-]`). This prevents path traversal and filename collisions.

18. What does `move_uploaded_file()` do and why is it used instead of `copy()`?

✓ It moves the temp upload to its final location while verifying the file was genuinely received through an HTTP POST upload, which adds a security check against forged file paths.

19. If the database insert fails after a video is saved to disk, what happens to the file?

✓ The catch block calls `@unlink($destination)` to delete the orphaned file, keeping disk and database consistent.

20. How does `search-players.php` build its filters dynamically yet safely?

✓ It pushes condition strings (e.g. `"pp.position_primary = ?"`) into a `$where` array and the matching values into a `$params` array, then joins with `AND`. Values still go through placeholders, so it stays injection-safe.

21. How is the age filter converted into a database condition?

✓ It converts an age into a date boundary with `date('Y-m-d', strtotime('-N years'))` and compares against `pp.date_of_birth` (e.g. `age_min` → `date_of_birth <= maxDob`).

22. How is pagination implemented in the search query?

✓ `$perPage` is clamped with `min(50, max(1, ...))`, `$offset = ($page-1)*$perPage`, and the query ends with `LIMIT ? OFFSET ?`. A separate `COUNT(*)` query gives the total used to compute pages.

23. What CORS / preflight handling appears at the top of each endpoint?

✓ They send `Access-Control-Allow-Origin/Credentials/Methods/Headers` headers and short-circuit the `OPTIONS` preflight request with `http_response_code(204); exit;`.

24. What does `logAction()` record and where? Give an example from club verification.

✓ It inserts into `system_logs` (actor, action, target type/id, and a JSON `meta`). For example, verifying a club logs `'club_verified'`; rejecting logs `'club_rejected'` with the reason in meta.

25. How does the server avoid leaking internal errors to the client?

✓ In `catch (PDOException $e)` blocks it writes details to `error_log()` but returns only a generic message like "A server error occurred." with HTTP 500, so stack traces and SQL details aren't exposed.

Section 2 — SQL

Questions 26–50

Covers `db/schema.sql` (11 tables) and the queries used by the PHP endpoints.

26. Why is the database created with `CHARACTER SET utf8mb4` and `utf8mb4_unicode_ci`?

✓ `utf8mb4` stores the full Unicode range including Arabic text and emoji (4-byte characters), and `unicode_ci` gives case-insensitive, language-aware sorting/comparison.

27. Why is the `InnoDB` engine chosen over `MyISAM`?

✓ `InnoDB` supports transactions, row-level locking, and **foreign-key constraints** — all of which this schema relies on (e.g. the registration transaction and the FK relationships between tables).

28. What does `INT UNSIGNED AUTO_INCREMENT PRIMARY KEY` give each table?

✓ A non-negative integer ID that the database auto-generates and increments for every new row, serving as the unique primary key.

29. Why is the `users.email` column declared `VARCHAR(191) NOT NULL UNIQUE`?

✓ `UNIQUE` stops duplicate accounts; 191 is the classic max length for a fully-indexed `utf8mb4` column under the older 767-byte index limit.

30. What is the purpose of the `role ENUM('player','club','admin')` column?

✓ It restricts the value to one of three allowed roles at the database level, preventing invalid roles and driving the authorization logic.

31. How does the schema implement "soft delete"?

✓ With an `is_deleted TINYINT(1) DEFAULT 0` flag (and `is_active` for suspension). Rows are marked deleted/inactive and filtered out in queries (`u.is_deleted = 0`) rather than physically removed.

32. What do `created_at` and `updated_at` default to, and what makes `updated_at` special?

✓ Both default to `CURRENT_TIMESTAMP`. `updated_at` also has `ON UPDATE CURRENT_TIMESTAMP`, so MySQL automatically refreshes it whenever the row changes.

33. Why are `photo_url` and `logo_url` declared as `MEDIUMTEXT` instead of `VARCHAR`?

✓ Because images can be stored as base64 data URLs, which are very large. `MEDIUMTEXT` holds up to ~16 MB, far beyond a `VARCHAR` limit.

34. How is a player's position constrained, and what positions are allowed?

✓ By an `ENUM('GK','CB','LB','RB','CDM','CM','CAM','LW','RW','ST')` on `position_primary` (and a nullable `position_secondary`), limiting input to valid football positions.

35. What is the relationship between `player_profiles` and `users`, and how is it enforced?

✓ One-to-one: `player_profiles.user_id` is `NOT NULL UNIQUE` with `FOREIGN KEY (user_id) REFERENCES users(id)`, so each user has at most one profile and it must reference a real user.

36. Why do the skill columns use `TINYINT UNSIGNED DEFAULT 50`?

✓ `TINYINT UNSIGNED` holds 0–255 (skills are 0–100) using just one byte, and the default of 50 gives every new player a neutral mid rating.

37. What does the `UNIQUE KEY unique_request (club_id, player_id)` on `contact_requests` guarantee?

✓ A composite unique constraint — a given club can have only one contact request per player, preventing duplicate requests.

38. What is the purpose of the indexes such as `idx_position`, `idx_nationality`, and `idx_status`?

✓ They speed up the filtered search and listing queries (WHERE / ORDER BY on those columns) by avoiding full-table scans.

39. Why does `contact_requests` have both a `sent_at` and a nullable `responded_at`?

✓ `sent_at` records when the club sent the request; `responded_at` stays NULL until the player accepts/declines, then it stores the response time.

40. How does the `club_profiles` table track verification?

✓ With `verification_status ENUM('pending', 'verified', 'rejected')` plus `verified_at`, `verified_by` (FK to the admin user) and `rejection_reason`.

41. The `club_profiles` table has two foreign keys to `users`. What are they?

✓ `user_id` (the club's own account) and `verified_by` (the admin who verified it). Both reference `users(id)`.

42. Why is the `meta` column in `system_logs` a `JSON` type?

✓ Different log actions carry different extra data (e.g. a rejection reason). A JSON column stores flexible, structured metadata without adding a column per action.

43. What does the composite index `idx_user_read (user_id, is_read)` on `notifications` optimize?

✓ Queries that fetch a single user's unread notifications (filtering by `user_id` and `is_read` together), which is the common access pattern.

44. What does `INSERT IGNORE` do in the admin seed statement?

✓ It inserts the default admin only if it doesn't already exist; if the unique email is already present, the duplicate-key error is ignored instead of failing the script.

45. Explain the JOIN used in the player search query.

✓ `FROM player_profiles pp JOIN users u ON u.id = pp.user_id` — it links each profile to its user so the query can filter on `u.is_active = 1 AND u.is_deleted = 0` while selecting profile fields.

46. How is the total number of search results counted for pagination?

✓ A separate `SELECT COUNT(*)` with the same WHERE clause, read via `$countStmt->fetchColumn()`; the page count is then `ceil($total / $perPage)`.

47. Why does the nationality filter use `LIKE ?` with `'%'.$value.'%'` instead of `=`?

✓ To allow partial / substring matching on nationality. The wildcards are added to the bound parameter, not the SQL, so it remains a safe prepared statement.

48. What does `ORDER BY pp.updated_at DESC` achieve in the search results?

✓ It shows the most recently updated player profiles first, surfacing the freshest/most active players at the top.

49. What is `DATE` vs `DATETIME` in this schema — give an example of each.

✓ `DATE` stores a calendar day only (`date_of_birth`); `DATETIME` stores date + time (`created_at` , `last_login` , `verified_at`).

50. What guarantees referential integrity when a contact request or saved player is created?

✓ The foreign keys `club_id REFERENCES club_profiles(id)` and `player_id REFERENCES player_profiles(id)` — the database rejects rows that point to non-existent clubs or players.

Section 3 — JavaScript

Questions 51–75

Covers `js/storage.js` , `js/auth.js` , `js/components.js` , `js/club.js` , and `js/player.js` .

51. What is the role of the `apiRequest()` function?

✓ It is the central HTTP wrapper around `fetch()` . It attaches headers, serializes the body to JSON, parses the response, throws on non-OK status, and (in `storage.js`) auto-stores the JWT token.

52. In `storage.js` , how is the JWT auth token sent with each request?

✓ It reads the token from `localStorage` via `GOG_DB.getToken()` and, if present, sets the header `Authorization: 'Bearer ' + token` .

53. How does `apiRequest` convert a `FormData` object (e.g. a file upload) into JSON?

✓ It iterates `body.entries()` ; any `File` value is read into a base64 data URL via `_fileToDataURL()` , other values are copied as-is, then the object is `JSON.stringify` -ed with `Content-Type: application/json` .

54. How does `_fileToDataURL()` wrap the `FileReader` API in a Promise?

✓ It returns `new Promise((resolve, reject) => {...})` , sets `reader.onload` to resolve with `e.target.result` and `reader.onerror` to reject, then calls `reader.readAsDataURL(file)` — turning a callback API into an awaitable one.

55. How does `apiRequest` turn an HTTP error response into a thrown error?

✓ After parsing JSON it checks `if (!response.ok)` and throws `new Error(data.message || ...)` , so callers can use `try/catch` to show a toast.

56. How does `requireAuth()` in `storage.js` guard a page?

✓ It checks for a token, calls `/api/auth/me` , verifies the returned `role` is in the allowed list, and redirects to `/login.html` if the token is missing, invalid, or the role doesn't match.

57. Why is `escapeHtml()` critical, and how does it work?

✓ It prevents XSS when user data is injected via `innerHTML` . It replaces `& < > " '` with their HTML entities so user content can't inject tags or scripts.

58. How is a player's age calculated on the client?

✓ `Math.floor((Date.now() - new Date(dob)) / 3.156e10)` — the millisecond difference divided by the number of ms in a year ($\sim 3.156 \times 10^{10}$).

59. How does `handleLogin()` redirect users after a successful login?

✓ It inspects `res.role` and sets `window.location.href` to the player, club, or admin dashboard accordingly (with a small `setTimeout` so the success toast is visible).

60. How does the player registration form validate before sending?

✓ It checks all required fields are filled, that `password === confirm_password`, and that the password is at least 8 characters — showing a toast and returning early if any check fails.

61. How does `club.js` build the search query string from the filter inputs?

✓ It reads each filter input value, then uses `new URLSearchParams({ page, per_page, ...(position && { position }), ... })` — the conditional spread adds a key only when the value is truthy.

62. What JavaScript feature is `...(position && { position })` and what does it do?

✓ It is conditional object spread. If `position` is truthy it spreads `{ position }` into the object; if falsy, `false` spreads nothing, so empty filters are omitted from the query.

63. How does `loadPlayerPublicProfile()` load profile, skills, career and videos efficiently?

✓ It runs all four `apiRequest` calls in parallel with `await Promise.all([...])` instead of awaiting them one after another, reducing total load time.

64. How is the optional contact message captured before sending a request?

✓ `promptMessage()` returns a Promise that opens a modal; the "send" button resolves with the textarea value, and "cancel" resolves `null`, which the caller treats as a cancellation.

65. How does `renderToast()` auto-dismiss a notification?

✓ After appending the toast it uses `setTimeout` to add a `hide` class, listens for `animationend` to remove the element, and has a fallback `setTimeout(... remove ...)`.

66. How does `renderModal()` wire up its footer buttons and close behavior?

✓ It builds buttons from the `actions` array, attaches each `onClick` via `addEventListener`, closes on the × button, and closes when the user clicks the overlay backdrop (`e.target === overlay`).

67. How can the user close any modal with the keyboard?

✓ A global listener `document.addEventListener('keydown', e => { if (e.key === 'Escape') closeModal(); })` closes the active modal on the Escape key.

68. How does `renderNavbar(role)` produce different navigation for each role?

✓ It keeps `links` and `actions` objects keyed by role (guest/player/club/admin), picks the matching entry, builds the HTML, and replaces the `#navbar-placeholder` element.

69. How does `positionBadge()` decide the badge color class?

✓ It checks the position against the `GK_POSITIONS`, `DEF_POSITIONS`, `MID_POSITIONS`, `FWD_POSITIONS` arrays and returns a span with class `badge--gk/def/mid/fwd`.

70. How does the photo upload preview work in `player.js` without uploading immediately?

✓ On `change` it validates the file is an image under 5 MB, then a `FileReader.readAsDataURL` loads it and sets the preview `img.src` to the resulting data URL.

71. How are career-history rows collected from the dynamic form?

✓ `collectCareerRows()` selects all `.career-row` elements, maps each to an object (club, season, appearances, goals, assists with `parseInt(...) || 0`), and filters out rows with no club name.

72. What does the optional chaining operator (`?.`) do, with an example from the code?

✓ It safely accesses a property/method only if the left side isn't null/undefined, e.g. `btn?.closest('.player-card')?.remove()` — avoids "cannot read property of null" errors.

73. Why does almost every JS file start with `'use strict';` ?

✓ Strict mode catches silent errors (e.g. assigning to undeclared variables), forbids some unsafe syntax, and makes the code more predictable and secure.

74. Why is page-init code wrapped in `document.addEventListener('DOMContentLoaded', ...)` ?

✓ It ensures the DOM is fully parsed before scripts query elements or attach event listeners, preventing "element is null" errors when the script runs before the markup exists.

75. How is the page-specific logic selected in `club.js` ?

✓ It reads `document.body.dataset.page` (the `data-page` attribute) and branches — e.g. `club-dashboard` runs the search, `view-player` loads the public profile.

Section 4 — HTML & CSS

Questions 76–100

Covers `index.html` , `login.html` , `css/variables.css` , `css/base.css` , and the design system.

76. What do `lang="ar"` and `dir="rtl"` on the `<html>` tag do?

✓ They declare the page language as Arabic and set right-to-left text direction, so the whole layout, text flow, and alignment mirror correctly for Arabic.

77. What is the purpose of `<meta name="viewport" content="width=device-width, initial-scale=1.0">` ?

✓ It makes the page responsive by matching the viewport to the device width and setting the initial zoom to 1, so mobile devices don't render a zoomed-out desktop layout.

78. Why is `<meta charset="UTF-8">` essential for this project specifically?

✓ The interface is in Arabic; UTF-8 ensures Arabic characters and emoji are decoded and displayed correctly.

79. What are the Open Graph (`og:`) meta tags used for?

✓ They control how the page appears when shared on social media — the title, description, and type shown in the link preview card.

80. Why does `login.html` include `<meta name="robots" content="noindex">` ?

✓ To tell search engines not to index the login page, keeping it out of public search results.

81. What do the `preconnect` link tags to Google Fonts accomplish?

✓ They open early network connections to the font servers so the Cairo/Bebas Neue fonts download faster, improving perceived load performance.

82. How does the project implement a design-token / theming system in CSS?

✓ Via CSS custom properties defined on `:root` in `variables.css` (colors, spacing, radii, shadows, fonts, z-index), used everywhere with `var(--token)` . Changing one variable restyles the whole site.

83. Give an example of using a CSS variable and explain `var()` .

✓ e.g. `color: var(--text-primary);` or `padding: var(--sp-8);` . `var()` substitutes the value of the named custom property, with an optional fallback.

84. What does the universal reset `*, *::before, *::after { box-sizing: border-box; margin:0; padding:0; }` do?

✓ It removes default browser margins/padding and makes width/height include padding and border, so element sizing is predictable across the whole site.

85. What does `font-size: clamp(2rem, 5vw, 3.5rem)` on `h1` achieve?

✓ Responsive (fluid) typography: the size scales with the viewport (`5vw`) but never goes below 2rem or above 3.5rem — no media queries needed.

86. How is the shimmering gold gradient text on the logo created?

✓ A `linear-gradient` background is clipped to the text with `-webkit-background-clip: text` and `-webkit-text-fill-color: transparent` , then animated by shifting `background-position` via the `shimmer` keyframes.

87. How are CSS animations defined and reused here?

✓ With `@keyframes` (e.g. `fadeSlideUp` , `shimmer` , `spin` , `pulse`) applied through utility classes like `.animate-fade-up` , with staggered `.delay-1..5` classes setting `animation-delay` .

88. How does `components.js` stagger card entrance animations?

✓ On `DOMContentLoaded` it loops over `.animate-fade-up` elements and sets each one's `style.animationDelay = ${i * 0.05}s` , so cards fade in one after another.

89. What accessibility features appear in the "How it works" tabs on the home page?

✓ ARIA roles and states: `role="tablist"` , `role="tab"` , `aria-selected` , `aria-controls` , and `role="tabpanel"` with `aria-labelledby` — describing the widget to screen readers.

90. What does the `.sr-only` utility class do?

✓ It visually hides an element (1px clipped box) while keeping it available to screen readers — used for accessible labels that shouldn't be seen.

91. How does the modal markup support accessibility?

✓ It uses `role="dialog"` , `aria-modal="true"` , and `aria-labelledby="modal-title"` , plus an `aria-label` on the close button.

92. How is a responsive card grid built without media queries?

✓ `.grid-auto { display:grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); }` — columns auto-fit to the container, each at least 280px wide, wrapping as space allows.

93. What is the `#navbar-placeholder` pattern and why is it used?

✓ Each page has an empty `<div id="navbar-placeholder">` that JavaScript (`renderNavbar`) replaces with the role-appropriate navbar, so the nav markup lives in one shared function instead of being duplicated in every file.

94. How do the responsive utility classes `.hide-mobile` / `.hide-desktop` work?

✓ Media queries: `@media (max-width:768px) .hide-mobile { display:none }` and `@media (min-width:769px) .hide-desktop { display:none }` show/hide elements per screen size.

95. What does `:focus-visible { outline: 2px solid var(--gold-primary); }` improve?

✓ Keyboard accessibility — it shows a clear gold focus ring only when navigating via keyboard, without showing it on mouse clicks.

96. How is the browser scrollbar and text selection color customized?

✓ `::-webkit-scrollbar` rules style a slim gold scrollbar, and `::selection` sets a gold highlight with dark text when the user selects text.

97. Why does the login form use `novalidate` and `autocomplete` attributes?

✓ `novalidate` disables the browser's default validation so the custom JS validation/toasts take over; `autocomplete="email" / "current-password"` help browsers/password managers fill the fields.

98. Why is `dir="ltr"` set on the email and password inputs inside an RTL page?

✓ Emails and passwords are Latin/left-to-right text; forcing `ltr` on those inputs keeps the characters and cursor in the correct order even though the page is RTL.

99. What semantic HTML5 elements structure the pages, and why prefer them?

✓ `<header>/<nav>/<main>/<section>/<footer>` (e.g. `<main class="auth-page">`, `<section class="hero">`). They convey meaning to browsers, SEO, and assistive tech better than plain `<div>` s.

100. How does the password show/hide toggle work between HTML and JS?

✓ The HTML has a `.password-toggle` button next to the input; `initPasswordToggles()` switches the input's `type` between `password` and `text` and swaps the eye icon (👁️ / 🙈) on click.
